4.4 How PHP autoloading works

Before asking Composer to generate a loader, trace the PHP mechanism it uses. When code needs a class that has not been declared, PHP can call registered autoload callbacks. The callback receives the class name and can require its declaration file. PHP then continues the original operation.

Replace the manual requires temporarily

Keep the two class files from 4.1. Add `public/tutorials/4-4-php-autoloading.php`:

```php
<?php
declare(strict_types=1);

use App\Tutorials\Class4\Admin\User as AdminUser;
use App\Tutorials\Class4\User;

$loader = function (string $class): void {
    $prefix = 'App\\';
    if (!str_starts_with($class, $prefix)) {
        return;
    }

    $relativeClass = substr($class, strlen($prefix));
    $file = __DIR__ . '/../../src/'
        . str_replace('\\', '/', $relativeClass) . '.php';

    if (is_file($file)) {
        require_once $file;
    }
};

spl_autoload_register($loader);

echo (new User('Avery'))->getName(), '<br>', PHP_EOL;
echo (new AdminUser('Morgan'))->getName(), '<br>', PHP_EOL;
```

Run `php public/tutorials/4-4-php-autoloading.php`. Output is unchanged. Trace `App\Tutorials\Class4\User`: match the App prefix, remove it, translate `Models\User` to `Models/User.php`, and load it beneath `src/`. `__DIR__` anchors the lookup to the entry file. This narrowly scoped demonstration expects class names in our source code; it is not a general-purpose loader for user-supplied paths.

Make loading visible

Temporarily add `echo 'Loading ', $class, '<br>', PHP_EOL;` inside the `is_file` block before the require. Instantiate User a second time at the bottom. Expect one loading message for each of the two classes, even though three objects are created. Restore the file above afterward.

The callback loads declarations; it does not construct objects. The `use` statement alone does not trigger loading. If every registered loader declines and a requested class is still unavailable, construction fails with a class-not-found Error.

Name the relevant functions

Function	Role
`spl_autoload_register($loader)`	Add a callback to PHP's autoload queue
`spl_autoload_functions()`	Inspect the registered callbacks
`spl_autoload_unregister($loader)`	Remove that callback

Loaders should quietly decline names they do not handle so another loader can try. Class-like declarations, including interfaces and traits, can be autoloaded; standalone function calls do not trigger this mechanism. Legacy `__autoload()` was removed in PHP 8.0 and should not be used.

Composer generates a reusable class loader and registers its callback when `vendor/autoload.php` is required. We will replace this demonstration callback in 4.5 so Composer handles both our classes and installed libraries.

Reference: PHP autoloading documentation.